

Examen SecOS - (c) Airbus

L'examen consiste en la création d'un OS multi-tâches préemptif, en vous appuyant sur vos connaissances du cours et les mini TPs réalisés.

Vous êtes libres de modifier l'intégralité du noyau si vous le jugez nécessaire.

Objectifs

Permettre un ordonnancement préemptif de 2 tâches (T1 et T2) en ring 3 :

- Le code et les données des tâches seront inclus dans le noyau à la compilation, comme nous l'avons fait dans les TPs précédents pour la fonction `userland()`.
 - Chaque tâche sera représentée par une fonction (ex. `user1()`, `user2()`) exécutant une boucle infinie.
 - La pagination doit être activée:
 - Le noyau est identity mappé
 - Les tâches sont identity mappées
 - Les tâches possèdent leurs propres PGD/PTB
 - Les tâches ont une zone de mémoire partagée :
 - De la taille d'une page (4KB)
 - À l'adresse physique de votre choix
 - À des adresses virtuelles différentes
 - Les tâches doivent avoir leur propre pile noyau (4KB)
 - Les tâches doivent avoir leur propre pile utilisateur (4KB)
 - La tâche 1 écrit un compteur (`uint32_t`) dans la zone de mémoire partagée
 - La tâche 2 affiche le compteur depuis la zone de mémoire partagée
 - L'affichage dans la tâche 2 s'effectuera à l'aide d'un appel système :
 - L'interface utilisateur :
 - A le prototype: `void sys_counter(uint32_t *counter);`
 - Notez bien que l'argument est une adresse virtuelle ring 3
 - L'interface noyau :
 - Est installée à l'interruption `0x80`
 - Reçoit un pointeur utilisateur de type `uint32_t*`
 - S'occupe de l'affichage du compteur (via `debug()`)
 - Le gestionnaire de l'interruption 32 (`irq0` = horloge) doit vous permettre à chaque interruption de passer de la tâche 1 à la tâche 2 et inversement. Il doit également savoir détecter s'il a interrompu le noyau ou une tâche utilisateur.
 - Une fois la mémoire, les interruptions et les tâches initialisées, le noyau doit activer les interruptions matérielles et effectuer une boucle infinie.
-

Astuces

- Faites vous une cartographie mémoire complète pour vous y retrouver : . Adresses physiques noyau . Adresses physiques utilisateurs . Adresses virtuelles utilisateurs

- Vous pouvez modifier le linker script afin de situer votre code utilisateur à des adresses physiques pratiques (alignées sur une entrée de PDE par exemple (ex 4MB, 8MB). Ceci vous aidera à configurer le PGD des tâches ring 3. Pour cela il faut :
 - Créer une section ".user" dans linker.lds
 - Préfixer vos fonctions `user1()` et `user2()` avec l'attribut de section GCC `__attribute__((section(".user")))`.
 - Regardez "include/mbi.h" et "core/start.c" pour vous inspirer de la manière dont les en-têtes multiboot sont déclarées.
 - Vous pouvez également créer une section pour les données user. Cette section pourrait contenir les piles user, la page de mémoire partagée par exemple.
- Faites attention aux droits des PDEs/PTEs ... ils sont combinés !
- Les piles avancent à l'envers ! On part des adresses hautes.
- La première exécution d'une tâche est un peu délicate à mettre en oeuvre. Pensez bien au chemin d'exécution (retours successifs d'appels de fonctions) qu'elle va devoir parcourir avant d'atteindre l'instruction "iret" responsable de la bascule en ring 3. Il s'agit de correctement initialiser le contexte d'interruption initial de la tâche, au sommet de sa pile noyau.

À rendre

J'attends un patch qui compile et me permet de tester votre travail. Je vous conseille de "cloner" à nouveau le repo git de secos et de commencer l'examen depuis cette base saine dans une nouvelle branche (et non pas depuis votre working directory utilisé pour les mini TPs).

Note : Me partager, dans le format de votre choix (.md, .jpg, .pdf, etc.) une documentation succincte de vos choix de cartographie mémoire serait un plus.

Un exemple de workflow:

```
$ git clone https://github.com/agantet/secos-ng
$ cd secos
$ git checkout -b exam

$ emacs ... gcc ... rage !
$ emacs ... gcc ... fear !
$ emacs ... gcc ... rulz :)

$ git add <file>
$ git commit -m "exam finished"
$ git format-patch $(git merge-base --fork-point master)..exam
$ tar czf nom.prenom.tar.gz *.patch
```

Ceci vous génère une archive compressée contenant vos modifications, que vous pouvez m'envoyer par mail.